PWLLHELI SAILING CLUB

Pwllheli Race Officer

Relay & Front Door Guide

The one machine the club is reached through: front door, live camera and GPS tracking

Covers app version 1.000

Internal technical reference · human-supervised prototype

CONTENTS

What is in this guide

NOTE

The step-by-step procedures here mirror **deploy/live_stream/README.md** in the app repository, which stays the working copy next to the config files. If the two ever disagree, believe the README and re-build this guide (**python scripts/build_relay_guide.py**).

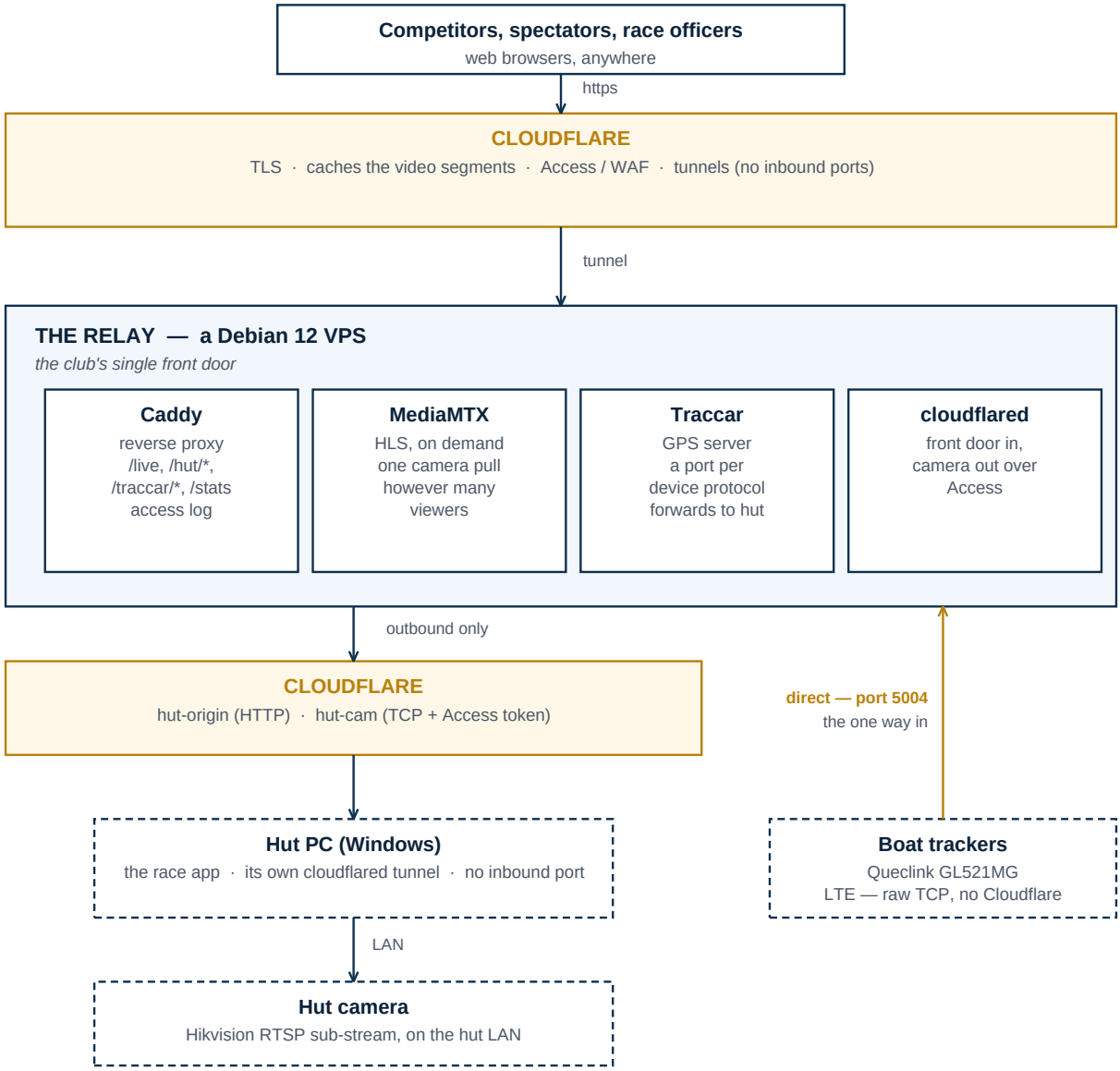
CHAPTER 1

What the relay is, and what it does

The relay is a small Debian 12 machine — a VPS — that the club rents. It began as a way to show the hut camera publicly without the off-grid hut uploading one video stream per viewer. It has since become the piece everything else depends on: it is the club's **single public front door**, it runs the **GPS tracking server** behind the live race map and the automatic finishes, and it is the one place where **every visitor is logged**.

Nothing reaches the club from the internet except through it. That is worth understanding before you change anything on it.

Note which way round the connections go. The relay does **not** reach into the hut: Caddy reverse-proxies to **hut-origin** over Cloudflare for the app, and cloudflared bridges the camera over Cloudflare Access — both go out to Cloudflare and come back down the *hut PC's own* tunnel, which is why the hut needs no open port and the camera is never exposed. The one exception is the **boat trackers**: they open a raw TCP socket straight to the relay, because there is no tunnel client on a tracker to carry it. That single link is the only inbound access in the whole system.



How the whole system fits together. The relay does not reach the hut directly: the app and the camera are both behind Cloudflare and the hut PC's own tunnel. Only the boat trackers connect straight to the relay — which is why they are the one thing needing an open port.

The four jobs it does

Front door	**pro.pwllhelisailingclub.org** resolves to the relay, and Caddy proxies to the race app on the hut PC. When the hut is offline it serves a holding page instead of a browser error, so the public address never simply fails.
Live camera	MediaMTX serves the camera as HLS at **/live**, pulling from the hut **only while someone is watching** and only **once** however many are watching. Cloudflare caches the video segments, so the relay's uplink stays flat as viewers grow. Club and sponsor logos are burned in on the relay from the app's own branding settings.

GPS tracking	Traccar receives the boats' tracker reports, and both answers the hut app's polls and pushes each new fix to it. This is what the live race map, the position-on-the-water list and the automated finishes run on.
Visitor logging	Because every request passes through Caddy, one access log covers the race app, the competitor pages and the stream — with the real visitor IP rather than the tunnel's.

IMPORTANT

The trade-off worth being honest about. Folding the old separate front-door PC into the relay means a single machine now carries the whole public face of the club. If the **hut** is down, visitors get the holding page and the live camera still works. If the **relay** is down, **pro.pwllhelisailingclub.org** is unreachable altogether — race app, competitor pages, live stream and GPS tracking at once. The race office itself keeps working (the app runs on the hut PC and is reachable at **http://localhost:5050** there), so racing is never blocked; it is the public view and the tracking that stop.

Three machines

The camera	A Hikvision IP camera on the hut LAN. It is never exposed directly; the hut PC publishes its RTSP sub-stream as a TCP hostname on its own tunnel, locked to the relay with a Cloudflare Access service token.
The hut PC	Windows, in the start hut. Runs the race app under Waitress and its own cloudflared tunnel (**hut-origin**). Opens **no inbound ports**.
The relay	This guide. A Debian 12 VPS: roughly 1 vCPU, 512 MB–1 GB RAM and 4 GB of disk is enough. It can equally be a container on a home server — see Chapter 3.

CHAPTER 2

Before you start: accounts, hostnames, hardware

You will need

- The race app running on the hut PC, **v0.140 or later** (it provides the live branding feed and the public live-stream setting) — and v0.189 or later to use push tracking.
- A **Cloudflare account** managing **pwllhelisailingclub.org**, with the hut already reachable through a Cloudflare Tunnel.
- A **Debian 12** machine for the relay itself — a VPS, or a container on a home server (Chapter 3 covers both).
- Router access, if you want GPS tracking — it needs one forwarded port (Chapter 6).

Hostnames

Hostname	What it is	Type
pro.pwllhelisailingclub.org	The public front door. Points at the relay's tunnel; everything the public uses lives under it.	Tunnel (CNAME), proxied
hut-origin.pwllhelisailingclub.org	The hut PC's own tunnel, carrying the race app. The relay proxies to it; the public never uses it directly.	Tunnel (CNAME), proxied
hut-cam.pwllhelisailingclub.org	The camera's RTSP, published as TCP on the hut tunnel and locked to the relay with an Access service token.	Tunnel (CNAME), proxied
track.pwllhelisailingclub.org	Where the boat trackers report. This one is **not** a tunnel — trackers open a raw TCP socket, which a tunnel cannot carry.	DNS only, to your public IP
traccar.pwllhelisailingclub.org	Optional. Traccar's own web interface, if you choose to publish it rather than reach it over the LAN or SSH.	Tunnel (CNAME), proxied

NOTE

track. and **traccar.** cannot be the same name. A tunnel hostname has to be a CNAME to the tunnel, while the trackers need a plain A record pointing at your public IP for the forwarded port — one name cannot be both.

CHAPTER 3

Building the relay

Anything running **Debian 12** with about 1 vCPU, 512 MB–1 GB of RAM and 4 GB of disk will do. There are two ways to host it, and they differ only in where the machine lives.

A VPS	**Recommended, and what the club runs.** It has a public IP of its own, so the trackers reach it directly, and nothing depends on a home connection or router. Not sharing an uplink with a household is worth a good deal on a race day.
A home server	A Debian 12 LXC on Proxmox — unprivileged, start on boot, no nesting needed. Free, and a perfectly good way to start out. The costs are that the trackers need a port forwarded on the home router, and the club's whole public face then rides on the home broadband.

Everything that follows is the same either way. **lxc/setup.sh** is a plain Debian install script despite the folder name, and the systemd units, Caddyfile and MediaMTX config are identical. As root on the relay:

1. Get the **deploy/live_stream/** folder from the app repository onto the machine — clone it, or on Proxmox push it from the host with **pct push <ctid> -r /path/to/deploy/live_stream /root/live_stream**.
2. Run the installer. It installs MediaMTX, Caddy, cloudflared, ffmpeg, Python and Traccar, copies the config and web files into **/opt/relay**, writes **/etc/relay/relay.env** and installs the systemd services.

```
cd /root/live_stream && bash lxc/setup.sh
```

Then fill in **/etc/relay/relay.env**. The tunnel token comes from Chapter 4 and the camera Access token from Chapter 5, so expect to come back to this file:

```
CAMERA_URL=rtsp://USER:PASS@localhost:18554/Streaming/Channels/102
MANIFEST_URL=https://pro.pwllhelisailingclub.org/api/branding/live
CAMERA_FPS=15
TUNNEL_TOKEN=<the pro. tunnel token, Chapter 4>
CAM_HOSTNAME=hut-cam.pwllhelisailingclub.org
CAM_ACCESS_ID=<Access service token Client ID, Chapter 5>
CAM_ACCESS_SECRET=<Access service token Client Secret, Chapter 5>
```

Start everything:

```
systemctl restart mediamtx cloudflared-tunnel cloudflared-camera
systemctl reload caddy
```

All the services start on boot. There are five to know about:

caddy	The front door and reverse proxy. Reload after any Caddyfile change.
mediamtx	The HLS server; starts the branded camera pull on demand.

cloudflared-tunnel	Brings **pro.** in from Cloudflare to Caddy on port 80.
cloudflared-camera	Opens the hut camera as **localhost:18554** on the relay.
traccar	The GPS tracking server (Chapter 6). Only if you use tracking.

```
journalctl -u mediamtx -f # follow any service's log
systemctl status caddy
```

CHAPTER 4

Cloudflare: the front door and caching

Point the front door at the relay

On the relay's dashboard-managed tunnel, add a **Public Hostname**: **pro.pwllhelisailingclub.org**, type **HTTP**, URL **http://localhost:80**. Copy that tunnel's **token** into **TUNNEL_TOKEN** in relay.env. If you are migrating from a separate front-door machine, remove **pro.** from it first — two tunnels advertising the same hostname is a confusing failure to debug.

Cache the video, and only the video

Edge caching is what makes the stream scale: without it every viewer's segment requests come back to the relay. Add two Cache Rules under **Caching** → **Cache Rules**:

Segments	Match **.ts**, **.mp4** and **.m4s** paths on the host → **Eligible for cache**, **Edge TTL: Override origin** → **10 seconds**.
Playlist	Match **.m3u8** on the host → **Bypass cache**. The playlist changes every second; a cached one freezes the stream.

Optionally turn on **Smart Tiered Cache** for extra origin offload. Make sure **Development Mode is off** — it forces every response to bypass the cache, and it is easy to leave on after debugging.

IMPORTANT

Never add a broad "Cache Everything" rule for pro.pwllhelisailingclub.org. Keep every cache rule scoped to the video suffixes above. The race app serves its pages with **Cache-Control: no-store** — the login page in particular — and Caddy passes that straight through, so Cloudflare leaves them alone by default. A "Cache Everything" rule with **Edge TTL: Override origin** ignores **no-store** and will serve a cached login page *without* its matching session cookie, which breaks signing in with *"Bad request: CSRF token missing or invalid"* for the next visitor.

Checking it works

Open the stream, then in the browser's developer tools look at the response headers of a **.ts** request. After the first **MISS** you want **cf-cache-status: HIT**. Once segments are hitting, extra viewers cost the relay nothing.

CHAPTER 5

The live camera stream

Camera settings that matter

In the camera's web interface, on the **sub-stream** (the relay deliberately uses the small one):

- **H.264**, not H.265 — H.264 plays in every browser with no transcoding on the relay.
- Around 640×480–704×576, 512–1024 kbps, 12–15 fps.
- **I-frame interval set equal to the frame rate** (e.g. 15). This is the single biggest latency lever: HLS can only cut a segment on a keyframe, so the Hikvision default of 50–100 gives 10 s or more of delay.

Create a **live-view-only camera user** for the relay rather than using the admin account.

Publishing the camera to the relay

The camera stays on the hut LAN. On the hut PC's existing cloudflared tunnel, add a public hostname **hut-cam** of type **TCP** pointing at **CAM_IP:554**. Then secure it: Zero Trust → Access → Applications → a self-hosted app for that hostname with a **service-token** policy. Put the token's Client ID and Secret into relay.env. Without that policy the camera's RTSP port is effectively public.

How the stream behaves

- The camera is pulled **only while someone is watching**, and the pull stops about 20 seconds after the last viewer leaves.
- However many people watch, there is still **one** pull from the hut.
- Club and sponsor logos are burned in on the relay, read from the app's own branding settings — change the logos in the app, not on the relay. The manifest is re-read each time the stream starts, so restart the stream to pick up new logos.

> **NOTE**
>
> Turn it on in the app afterwards: **Settings** → **Video recording** → **Public live stream URL** = **https://pro.pwllhelisailingclub.org/live**. That is what makes the competitor pages show live video rather than refreshing snapshots. The relay's watch page is embedded directly in those pages, so **copy site/index.html to /opt/relay/site/ whenever it changes** — an older copy does not report that it is playing, and the app's panels stay on stills for ever.

CHAPTER 6

GPS tracking with Traccar

Boats carry Queclink **GL521MG** LTE trackers. They report to Traccar on this relay, and Traccar gets their positions to the hut app two ways at once — it answers the app's polls, and it pushes each fix as it decodes it. The live race map, the position-on-the-water list and the automated finishes all run on this.

Traccar is installed by **lxc/setup.sh**. It serves its web interface and REST API on port **8082**, and listens for devices on **a separate port per protocol** — one Traccar, many listeners. The port you need is the one for your device's protocol:

Device	Traccar protocol	Port
Queclink **GL521MG**, and the rest of the GL/GV series	gl200	**5004/tcp**
Teltonika **RUTX50** router, FMB/FMC series	teltonika	5027/tcp
Traccar Client phone app, simulate_trackers.py	osmand	5055 (HTTP)

IMPORTANT

5027 is Teltonika, not Queclink. Guides before v0.197 said to use 5027 for the boat trackers — that is the port the club's RUTX50 test device uses, and it is **wrong for a GL521MG**. Point a Queclink tracker at 5027 and it reaches Traccar's Teltonika decoder, which cannot parse @Track messages: the tracker reports happily, nothing appears in Traccar, and nothing is logged anywhere to say why. Use **5004**. Traccar's own protocol reference has the port in its address — **traccar.org/protocol/5004-gl200/**.

The inbound port

GPS tracking is the only part of the system that needs an inbound port. A tracker opens a raw TCP socket, and there is no tunnel client on the device to carry it — which is why this one cannot go through Cloudflare like everything else.

- **On a VPS:** allow **5004/tcp** in the provider's firewall or security group *and* in the host firewall (**ufw allow 5004/tcp**), then point **track.pwllhelisailingclub.org** at the VPS's IP as a **DNS-only** A record — grey cloud, because Cloudflare cannot proxy a raw TCP socket.
- **On a home server:** forward **5004/tcp** on the router to the container, and point the same DNS-only record at your home public IP.

NOTE

Open one port per protocol you actually use, and leave the rest closed. The club currently has three: **5004** for the Queclink trackers, **5027** for the Teltonika RUTX50 on the committee boat, and **5055** for osmand. This is the only inbound access anywhere in the system — the hut PC still opens nothing at all.

5055 is worth keeping open. It carries the OsmAnd protocol, and two useful things speak it. The free **Traccar Client** phone app is the first: set its server to **track.pwllhelisailingclub.org:5055**, give it a device identifier, add that identifier on the app's Trackers page, and a phone in a pocket becomes a tracker — the cheapest way to put a rescue RIB, the committee boat or a volunteer's boat on the chart

without waiting for GL521MG hardware or burning its battery. The second is **simulate_trackers.py**, which reports over the same protocol, so a whole simulated fleet can be sailed round the course through the real relay. It is plain HTTP with no authentication beyond the device identifier, so treat it as you do the other two: Traccar accepts only identifiers it knows, and nothing behind it is reachable from that port.

Provisioning the trackers

Point each tracker at **track.pwllhelisailingclub.org:5004** with the Queclink @Track configuration commands for the model (**AT+GTQSS** / **AT+GTSRI** on this family — check the GL521MG protocol document supplied with the units). The *Main server IP/domain* field accepts a hostname.

> **IMPORTANT**
>
> The GL521MG is a battery asset tracker designed for up to a year of standby, so its out-of-the-box reporting is far too coarse to order a finish. Set a **fast reporting rate for race days** — and plan to **charge the units between race days**, because a rate quick enough to time a line crossing costs a large multiple of the standby drain. They charge wirelessly (Qi).

Connecting the hut app

Caddy already exposes the REST API at **/traccar/***. Create an API token in Traccar (user settings), then in the app: **Settings** → **GPS tracking** → base URL **https://pro.pwllhelisailingclub.org/traccar**, paste the token, tick **Enable GPS tracking**, save. The status box should report how many trackers are reporting.

Pushing positions (recommended)

Polling costs up to a poll interval before the app sees a fix. That does not change a recorded finish *time* — crossings are interpolated between fix timestamps — but it does delay the **automatic horn** and what competitors are shown. Merge the entries from **traccar-forward.xml** into **/opt/traccar/conf/traccar.xml**, set the same secret as the **Push ingest token** in the app's Settings, and restart Traccar. Fixes then arrive the moment Traccar decodes them and finish detection runs on receipt.

```
# /opt/traccar/conf/traccar.xml (see traccar-forward.xml for the full block)
<entry key='forward.enable'>true</entry>
<entry key='forward.type'>json</entry>
<entry key='forward.url'>https://pro.pwllhelisailingclub.org/api/track/ingest</entry>
<entry key='forward.header'>X-RO-Track-Token: SAME_AS_THE_APP</entry>

systemctl restart traccar
```

> **NOTE**
>
> A token that does not match fails **silently** — the app carries on polling, every tracker reports, and the only symptom is a late horn. Check **Settings** → **GPS tracking**: the status box reads **Push: working — N fixes received, last just now** when it is right. Once push is working, raise the app's poll interval to about 30 seconds; polling stays on as the safety net and back-fills anything missed while the app was down.

This is deliberately **not** Traccar's “forwarder-only” mode. Traccar keeps its database, so it keeps the device registry (needed to send a tracker commands, such as raising its reporting rate near the line), the history the app back-fills from, and its web interface.

Automatic enrolment

With **database.registerUnknown** enabled, a tracker that is switched on creates itself in Traccar and then appears under **Trackers seen on the network** on the app's Trackers page, where a race officer picks its boat and presses Add — no 15-digit IMEI to read off the device. The trade-off is that the tracker port is open to the internet, so anyone who connects to it can cause a device row to appear. Nothing is tracked until a race officer assigns it to a boat; delete strays in Traccar.

Reaching Traccar's own web interface

You rarely need it — devices, assignment and removal are all done on the app's Trackers page. It is for Traccar-side admin: users and API tokens. It does **not** work at **…/traccar/**; that page loads and then spins for ever, because Traccar's interface is a single-page app that requests its JavaScript from the **site root**, where those requests fall through to the hut app instead. Two ways that do work:

- **On the LAN or over SSH** — **http://<relay-ip>:8082**, or from anywhere **ssh -L 8082:localhost:8082 root@<relay>** and then **http://localhost:8082**. Simplest, and keeps the login page off the internet.
- **Its own hostname** — add **traccar.pwllhelisailingclub.org** as a second public hostname on the relay's tunnel pointing at **http://localhost:80**; the Caddyfile already has the matching site block, so **systemctl reload caddy** is all this end needs.

IMPORTANT

Publishing that hostname puts a login page on the internet. Put **Cloudflare Access** in front of it, and at the very least change Traccar's default admin password and switch Registration off under Traccar → Settings → Server.

CHAPTER 7

Knowing who visits: access logging

Because Caddy is the single front door, one access log covers everything — the race app, the competitor pages, the watch page and the stream. It is enabled in the Caddyfile and writes JSON to **/var/log/caddy/access.log**, rolled at 20 MiB and kept for about 90 days.

- A global **trusted_proxies** / **client_ip_headers CF-Connecting-IP** block makes each line record the **real visitor IP**. Without it every request logs as **127.0.0.1** — the cloudflared peer on loopback.
- The high-volume HLS **.ts** segments are skipped, to keep the log about page visits.
- **Counting video viewers:** Cloudflare caches the segments, so most never reach the relay. Playlists are not cached, so an active viewer's periodic **.m3u8** refresh does hit Caddy — count distinct client IPs on those. For headline totals and geography across cached traffic too, use Cloudflare's own analytics.

IMPORTANT

IP addresses are personal data under UK GDPR. Retention is bounded by the log's **roll_keep_for** setting, **/var/log/caddy/** should stay readable by root only, and the public pages should carry a short note that access is logged.

A dashboard at /stats (optional)

A GoAccess HTML report can be served through the same Caddy at **/stats**; a commented block is ready in the Caddyfile. You need **GoAccess 1.9.2 or newer built with GeoIP2** — Debian's packaged version is older, so install from the official GoAccess apt repository and check with **goaccess --version | grep -i geo**.

1. Copy **relay_stats.sh** to **/opt/relay/**, make it executable, and run it from cron (e.g. every 10 minutes). It reads the current and rolled logs and adapts to the GoAccess version it finds.
2. Optionally drop MaxMind GeoLite2 **.mmdb** files into **/opt/relay/geoip/** for Country, City and network panels.
3. Make a password with **caddy hash-password**, uncomment the **handle /stats*** block and paste the hash in, then **caddy validate** and **systemctl reload caddy**.

IMPORTANT

Do not leave **/stats** unauthenticated — it exposes visitor data. Cloudflare Access in front of the path is a stronger gate than basic auth, and can be used as well as it.

CHAPTER 8

Running it day to day

Routine commands

```
systemctl status caddy mediamtx traccar # is everything up?
journalctl -u mediamtx -f # watch the stream start on demand
journalctl -u traccar -f # watch trackers connect
systemctl reload caddy # after editing the Caddyfile
caddy validate --config /etc/caddy/Caddyfile # check it before reloading
```

After updating the app repository

Several files on the relay are copies of files in the app repository, and they do not update themselves. After pulling a new version, copy across whatever changed and restart the affected service:

File	Goes to	Then
site/index.html	/opt/relay/site/	nothing (served directly)
relay_branded_source.py	/opt/relay/	systemctl restart mediamtx
relay_startline.py, odm_detector.py	/opt/relay/	systemctl restart mediamtx
relay_wind.py	/opt/relay/	systemctl restart mediamtx
Caddyfile	/etc/caddy/Caddyfile	caddy validate, then systemctl reload caddy
mediamtx.yml	/opt/relay/	systemctl restart mediamtx
relay_stats.sh	/opt/relay/	nothing (runs from cron)

NOTE

The watch page is the one that bites. The app's camera panels wait for it to report that it is playing before swapping from snapshots to video; an old copy reports nothing, so the panels sit on still pictures and everything *looks* like a broken stream.

Health checks worth doing before a race day

Check	How	Expect
Front door	open https://pro.pwllhelisailingclub.org	the race app, not the holding page
Live stream	open /live	branded video within a few seconds

Check	How	Expect
On demand	journalctl -u mediamtx -f while opening it	*runOnDemand command started*, then *stream is available*
Edge caching	DevTools → a .ts response header	cf-cache-status: HIT after the first MISS
Tracking	app → Settings → GPS tracking	*N tracker(s) reporting*
Push	the same status box	*Push: working — N fixes received*

CHAPTER 9

Troubleshooting

These are the non-obvious things the working setup depends on. Nearly every fault below was met once and fixed by a specific setting, so check here before changing anything.

The stream

No video, stuck at "Connecting…"	HLS must be **mpegts**, not low-latency. Low-latency HLS uses chunked playlist delivery that Cloudflare will not proxy, so it stalls behind the CDN.
Playlist 404s / redirect loops	Caddy must proxy **/hut/* without** stripping the prefix — MediaMTX redirects to **/hut/index.m3u8** and a stripped prefix breaks it.
Player paused at 0:00 on a healthy stream	An old **site/index.html**. From v0.187 it uses hls.js first: Chrome claims it can play HLS natively on some builds and then cannot decode it. Copy the current file to the relay.
Segments show cf-cache-status: BYPASS	Cloudflare will not cache a response carrying **Set-Cookie** or **Cache-Control: private/no-cache**, and MediaMTX sends both — Caddy strips them on segments. Also confirm Development Mode is off.
"Timestamps are unset" / "Non-monotonous DTS"	The camera RTSP has no usable presentation timestamps; the source script fixes it with **-use_wallclock_as_timestamps 1**.
Tears down every ~30 s, heavy buffering	Publish to MediaMTX over **TCP** and give the input a buffer. Also do not loop the logo images at full frame rate.
"branding manifest fetch failed (403)"	Cloudflare's bot protection blocks the default Python user-agent; the script sends a normal one. Also check Public branding is enabled in the app.
"runOnDemand … timed out" in a loop	The branded source did not publish in time, so MediaMTX killed it mid-startup — the ffmpeg errors are the kill, not the cause. Confirm the camera itself is fine with a direct pull, then raise **runOnDemandStartTimeout**, or disable the start-line overlay to drop its extra frame grab.
"probe failed" / "Could not find codec parameters"	Newer ffmpeg gives up before reading the sub-stream's dimensions. The relay scripts pass **-analyzeduration 10M -probesize 10M** on every RTSP read — make sure /opt/relay has the current copies.

The front door

Holding page when the hut is up	The hut's own tunnel is down, or Caddy cannot reach **hut-origin**. Check the hut PC's cloudflared service.

“Bad request: CSRF token missing or invalid” at login	Something is caching the app's HTML. Look for a “Cache Everything” rule or Page Rule on the hostname and remove it (Chapter 4).
Caddy will not start	**:2019 address already in use** means another Caddy is running; **:80 permission denied** means it is being run by hand rather than as the packaged service.

Tracking

A tracker never appears	Without **database.registerUnknown**, Traccar silently rejects an id it does not know — the tracker gets no error and nothing appears anywhere. Either enable it, or add the device by IMEI first.
A device is missing from the app but visible in Traccar	It probably belongs to no Traccar user — which is exactly what auto-registration creates. Traccar's own interface hides those unless *All Devices* is switched on, and their positions are not returned to the app's token at all. Adopting it in the app claims it.
“Could not read Traccar: HTTP Error 403”	Cloudflare is bot-blocking the request. The app sends a normal user-agent from v0.167 — make sure the hut app is up to date.
Push never arrives	The **forward.header** secret and the app's **Push ingest token** do not match, or the app is older than v0.189 and returns 400 rather than 401.

CHAPTER 10

File reference

Everything below lives in **deploy/live_stream/** in the app repository.

README.md	The working copy of these procedures, kept next to the files it describes.
lxc/setup.sh	The installer: packages, /opt/relay, /etc/relay/relay.env, systemd units, Traccar.
lxc/systemd/	Unit files for mediamtx, cloudflared-tunnel and cloudflared-camera.
Caddyfile	The front door: **/live** watch page, **/hut/*** to MediaMTX, **/traccar/*** to Traccar's API, everything else proxied to the hut app with the holding-page fallback — plus the access log and the optional **/stats** block. Its **frame-ancestors** list controls who may embed the watch page; the app's own origins are in it, so reload Caddy after changing that line.
mediamtx.yml	MediaMTX: mpegts HLS with 1-second segments, on-demand source, stream dropped 20 s after the last viewer.
relay_branded_source.py	Pulls the camera, burns in the club and sponsor logos from the app's branding manifest, and publishes into MediaMTX. Falls back to a plain copy if branding is off or unreachable.
relay_startline.py, odm_detector.py, startline_config.json	The optional start-line overlay: a colour-based detector that finds the orange ODM buoy and draws the line from the pole base to it. Best-effort and **prototype** — if anything is missing or the buoy is not confidently found, the stream simply runs without the line. Detection runs once when the stream starts, so the line is fixed for that viewing session.
relay_wind.py	The optional wind readout: TWD, TWS and gust across the top of the picture, fetched from the hut's public weather endpoint and rewritten into a small file that ffmpeg re-reads as it runs, so the numbers change without restarting the stream. Off unless **WIND_OVERLAY=1**. **Blank rather than stale**: a sample older than two minutes, or one missing a direction or speed, draws nothing at all — a dropped link must not leave an old wind burned into live-looking footage. Every stream start logs which state it is in.
site/index.html	The watch page served at **/live**, with hls.js bundled locally. Also posts its playing state to the parent window, which is how the app's camera panels know when to swap from snapshots to video.
site/hut_offline.html	The holding page shown when the hut app is unreachable.

traccar-forward.xml	The Traccar entries to merge for push forwarding, with the reasoning and the optional auto-enrolment setting.
relay_stats.sh	Generates the GoAccess report for **/stats**.
docker/	An alternative Docker Compose stack, if you would rather not install on the host directly.

NOTE

The relay holds no race data. Everything it serves either comes from the hut app live, or is a config file in the repository — so rebuilding it is a matter of re-running **setup.sh** and restoring **/etc/relay/relay.env**. The one thing worth keeping a copy of is that env file, because it holds the tunnel and camera Access tokens. Traccar's own database (devices, users, tokens) lives on the relay and is **not** in the app's backups — if you use tracking, back up **/opt/traccar/data/** as well.